Azamat Turganbayev

Firmware Intern Application, WindBorne Systems
turga0607@gmail.com | **azamatturganbayev.com** | **github.com/aturganbayev**

A project you're proud of and why

I'm proudest of the automated tactile surface exploration system I built on a UR5 robot. I mounted and calibrated a 6-axis force/torque sensor, aligned a 3,000-point CAD model to the robot's base frame using ICP registration to within 2.2 mm RMS accuracy, and wrote the full URScript and Python pipeline to execute controlled press sequences while logging synchronized force and pose data. I'm proud of it because when the data initially didn't match the robot's physical behavior, I had to debug across the full stack, sensor hardware, electrical connections, and software timing, before finding a 50 ms communication buffering lag as the root cause. It taught me that real hardware problems rarely live in just one layer.

Your development setup and tools

I develop primarily in Python and C/C++ on Linux, using Git for version control. For robotics work, I use ROS2, Gazebo for simulation, and URScript for direct UR5 programming. For mechanical design, I use SolidWorks, and for control systems work, MATLAB/Simulink and QUARC for real-time control on physical hardware. I rely on oscilloscopes and multimeters for hardware-level debugging when software behavior doesn't match expectations.

Debugging approach for a randomly resetting PCB

I want to be honest that I have not personally debugged a randomly resetting PCB, my hardware debugging experience is at the sensor and actuator integration level rather than board-level firmware. That said, based on how I approach any intermittent hardware fault: I would first check power supply stability under load with an oscilloscope, since brownouts and voltage sag are the most common cause of unexplained resets. Next, I would check for watchdog timer triggers or unhandled exceptions in the firmware that could cause a silent reset, and look at whether the resets correlate with specific events like high current draw, temperature, or specific code paths. I would try to reproduce the issue reliably before making changes, since intermittent faults are often environmental, thermal, EMI, or power, rather than purely logical.

Your perspective on firmware engineering mistakes or controversial best practices

I don't have enough firmware-specific experience to offer a controversial opinion here, so I'll be honest rather than invent one. From robotics software more broadly, I've found that a common mistake is trusting simulation results too much before validating on real hardware. I've seen this in my own work: control loops that behaved perfectly in Gazebo or Simulink needed real tuning once deployed on physical actuators, because real friction, sensor noise, and timing jitter don't show up the same way in simulation. My bias is to get onto real hardware as early as possible, even with an imperfect setup, rather than polishing a simulated version first.

A sketchy firmware workaround you've implemented

I'll be honest that I haven't worked at the firmware level specifically, so I don't have a firmware workaround to describe. The closest equivalent from my robotics work: when I found a 50 ms communication buffering delay was causing my UR5 force and pose data to desynchronize, my first fix was a workaround, I manually offset the timestamps in post-processing to realign the streams, rather than fixing the underlying buffering behavior in the acquisition pipeline. It got the data usable immediately, but I later went back and adjusted the acquisition timing itself so the offset wasn't needed as a patch.